1. MyWeb URL

- **Live Site:** http://myweb.usf.edu/~christodoulos

2. Logical Data Model

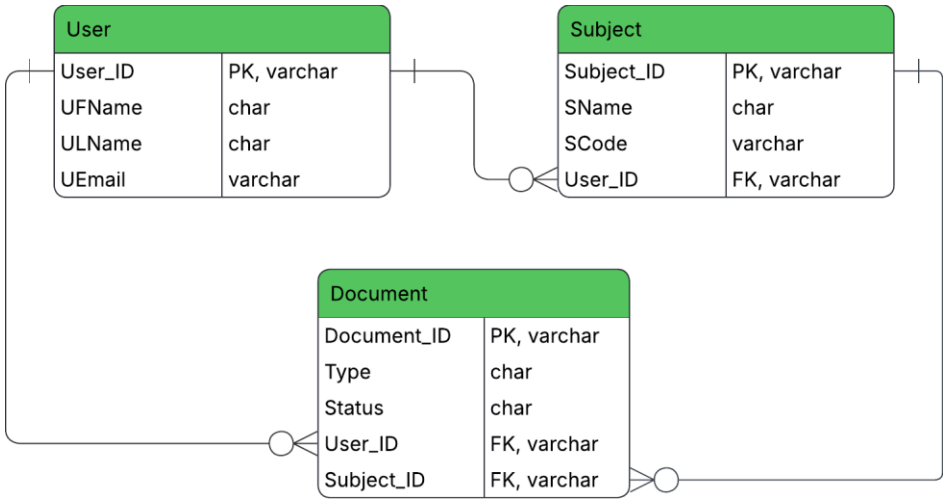


Figure 1: ERD

	A	B	C	D	E	F	G	H
1	Feature	Entity	PK	FK	Size	Null	Type	Description
2	User_ID	User	yes		255		varchar	user id
3	UFName	User			30		char	user first name
4	ULName	User			30		char	user last name
5	Uemail	User			100		varchar	user email
6	Document_ID	Document	yes		255		varchar	document id
7	Type	Document			20		char	document type (lecture,notes, recording)
8	Status	Document			10	yes	char	document status (reviewd, to be reviewed)
9	Description	Document			255	yes	char	document description
10	User_iD	Document		yes	255		varchar	user id
11	Subject_ID	Document	yes		255		varchar	subject id
12	Subject_ID	Subject	yes		255		varchar	subject id
13	Sname	Subject			50		char	subject name (eg App Dev for Analytics)
14	Scode	Subject			10	yes	varchar	subject code (eg ISM6225)
15	User_ID	Subject		yes	255		varchar	user id

Figure 2: Data Dictionary

Model Explanation: The StudyBuddy application relies on a hierarchical, one-to-many data model designed to organize academic materials efficiently. The core entities are User, Subject and Document.

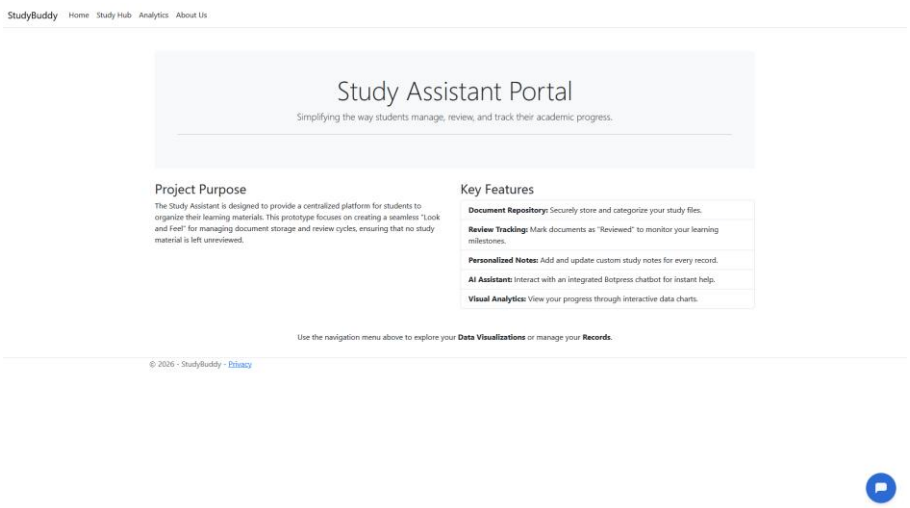
- **User Entity:** Represents the student. Since we are not tracking grades, or other data as this is just a personal folder that you can access from anywhere, the entity only has a Student First and Last name field and email address which could be used to log in to your account. User is connected to Document and Subject via 1:N connections as a user can choose to have 0 or many folders and can upload 0 or many files.

- **Subject Entity:** Represents academic subjects (e.g., ISM 6225, FIN 6406). It acts as the primary organizational container. It's also connected to Document in a 1:N relationship (a subject can have 0 or multiple documents.
- **Document Entity:** Represents individual study materials (PDFs, MP4s, DOCXs). Each document contains attributes for its title, file type, and status (Draft, In Progress, Reviewed).

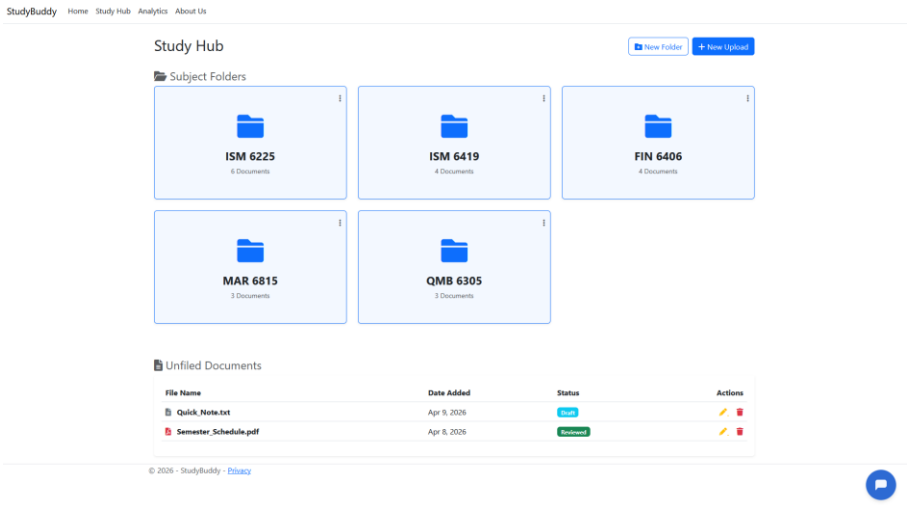
The logical relationship dictates that one Folder can contain many StudyDocuments, but each StudyDocument belongs to exactly one Folder (or defaults to an "Unfiled" state). This structure allows the front-end to dynamically cross-filter analytics and populate specific folder views based on the document's assigned subject attribute.

3. Screenshots

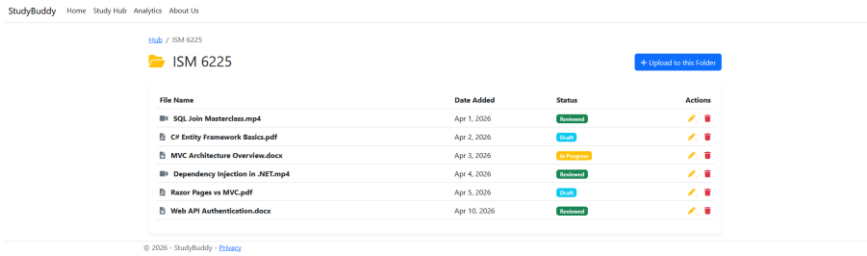
- Home Page



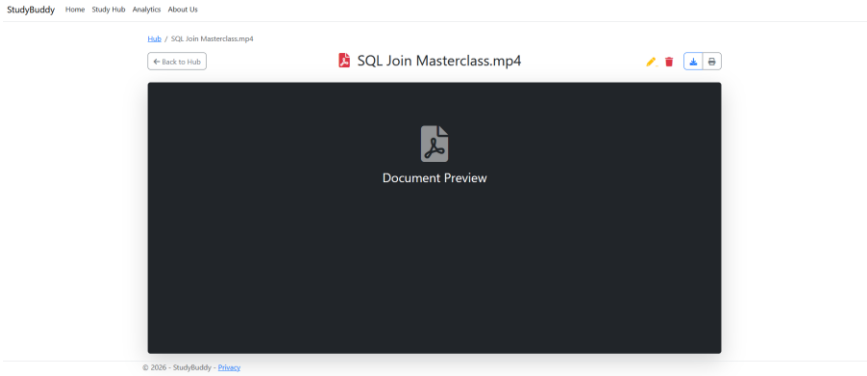
- Study Hub



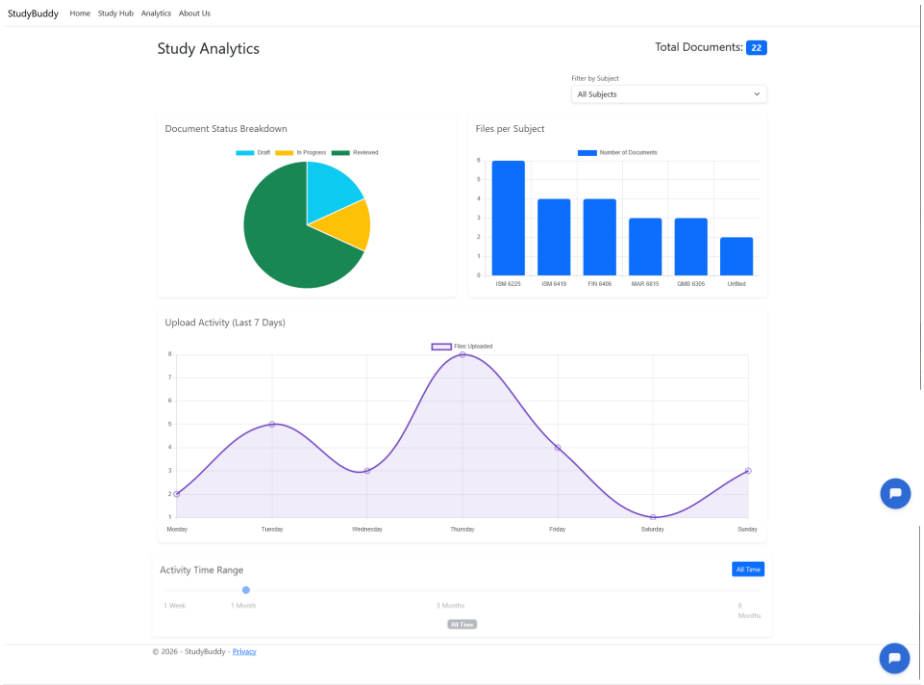
  - Folder View



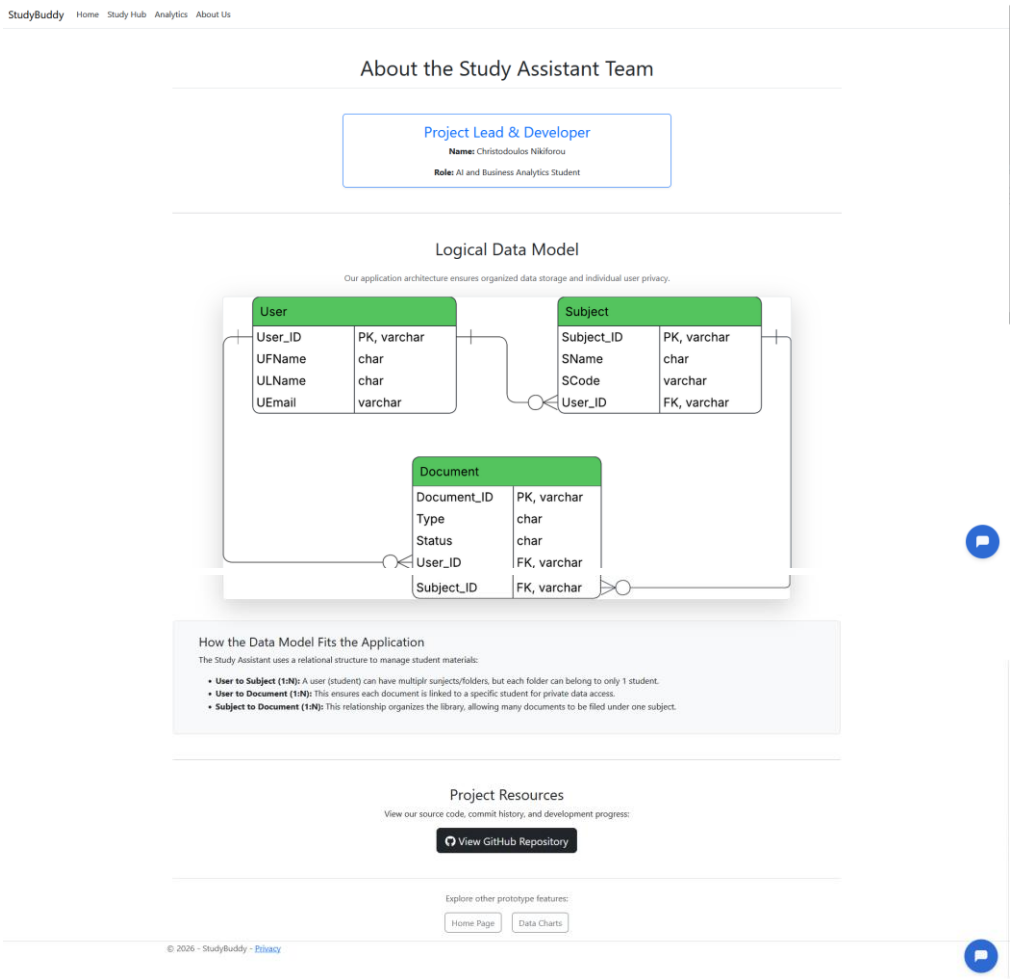
- Document View



- Analytics Dashboard



- About Us



4. Git Log Output

39a1177 - chris, 6 hours ago : - bug fixes - logic upgrades (analytics, data organization) - code testing and commenting
f4ec8b6 - chris, 19 hours ago : bug fixes and cosmetic changes
62f8c1b - chris, 20 hours ago : added chatbot adjusted some logic and data structure
918aaae - chris, 25 hours ago : added charts and analytics section
1da4921 - chris, 26 hours ago : - Implemented functianlity for all CRUD operations - Simulated some CRUD operations and user interactions - Improvements and refinements in code to add reusability and consistency accross the app
9292ac4 - chris, 30 hours ago : Main activity hub implementation Added: - Create (upload new/create folder) functionality - Read (view) functionality
38da6a6 - chris, 33 hours ago : Add project files.
bc60613 - chris, 33 hours ago : Add .gitattributes and .gitignore.

5. Self Reflection

I chose study hub as a website that would act as an assistant to students. The idea was for a private place where students can upload all their files and organize them. The site keeps

track of when the documents were uploaded, the documents can be stored in different folders and each document has a status; draft, in progress or reviewed, so the student can easily know how much work is done and what is left. There is also an analytics page which shows total documents uploaded and their current status, complete with filters to focus on a single subject. The website is complemented by a chatbot which can find files and keep track of what is going on, so the student is never overwhelmed.

Most of the challenges I faced were technical. Adding/expanding capabilities would sometimes break previously working code, and as the project got bigger it was a bit hard to keep track of what broke and how to fix it, as there are instances where at least 3 documents (eg. js, cshtml, css, and the base _layout or model classes) would try to work with each other. In addition, transferring the files to cyberduck was a challenge, since I had to convert from cshtml to plain html and update some of the code to make the website less dynamic and more static.

I used the help of AI to write a good part of the code and to help with debugging. I also made sure to create a data dictionary to supplement the ERD and kept my code files as separate and specific as possible, so I could locate and fix errors quicker, but also have clean readable code. In the end, I could've done a better job of separating js and html files, as I still have some small js scripts in a few html files.

The project took about 20-22 hours for me to bring to a satisfactory level, including debugging, uploading, testing and creating this document. My key learnings are that I should be more strict about separating js and html, and that I could spend a bit more time on planning out all the features that ended up being in the website I could maybe have less errors to deal with, compared to thinking of and implementing a new feature, or extending a feature on the fly.